Homework 2: Classify a Music Catalog

Pre-Registration Report Submission Deadline: **10/20 11:59 PM**
Final Report and Code Submission Deadline: **11/3 11:59 PM**

Introduction

In Homework 1, you audited a music catalog and explored the structure and geometry of its audio-feature space. The team now wants to use the catalog for a concrete purpose: **automatically tagging the genre of tracks that are added to the catalog in the future**.

You are given a *processed* subset of the same catalog with **5,300 tracks from 20 genres** and asked to build, evaluate, and choose classifiers for the task. As in a real catalog, the genres are not equally common: the largest and smallest genres have different numbers of tracks.

Before trusting any classifier, the team needs to answer questions such as:

- How does each classifier make its decisions, and how do its hyperparameters change what it learns?
- Why do different algorithms perform differently on the same data, and in which situations is each of them a good or bad choice?
- Is a single metric such as accuracy enough to judge a model performance, and if not, which metrics best align with the specific use cases?
- What does a reported score actually promise about tracks the model has never seen, and how should the test be designed so that the number is honest?

Your goal in this assignment is to answer these questions with evidence.

You will evaluate the classifier families covered in class: Nearest-neighbor classifiers, Bayesian classifiers, decision trees and their ensembles, and support vector machines.

What matters most is not writing the algorithms from scratch, but **designing the evaluation protocol** that every comparison depends on, comparing how the algorithms behave through each **model's inductive biases**, and **justifying when each approach is appropriate to use.** The purpose of the homework is **not to**

obtain the highest predictive accuracy, instead, you are expected to **evaluate each model honestly rather than evaluate through the flawed testing**.

You are expected to connect each conclusion to evidence, explain what you observe from how each algorithm and each metric is defined, and say clearly when the data do not support a conclusion. A weak or unsuccessful predictive result can therefore be a correct and important finding if you demonstrate it carefully and explain what it means.

The cleaned dataset has been provided to you. Do not add external datasets, external features, or information collected from other sources. Your conclusions should be supported by evidence from your analysis of the given dataset.

Rules

1. This is an individual assignment. You may discuss course concepts and general approaches with classmates, but the analysis you perform, the code you submit, and the writing in your report must be your own. Do not copy another student's code, results, figures, or written explanations. **Any submission found to contain plagiarism or copied work will receive a score of zero for the assignment.**

2. **AI tools**, including coding assistants and coding agents, are **permitted**. If you use them, you are still responsible for understanding, checking, and being able to explain or modify everything you submit. You must also submit the AI-use log described in Section 5.

3. **Write the report yourself.** Do not use AI tools to generate the content of your report. We want to read your own reasoning in your own words. Plain language is fine: a short, clear explanation of what you did and why is worth more than a polished paragraph you cannot defend at the demo. Reports that read as AI-generated will be checked at the demo, and you will be asked to explain each conclusion.

4. The report may be written in **either Traditional Chinese or English**. There are no specific requirements for the report format.

5. Google Colab is sufficient for this assignment to be accomplished, no GPU is required. You may use standard Python packages including

`pandas`, `numpy`, `scipy`, `matplotlib`, and `scikit-learn`. There are **no restrictions on the Python libraries** you may use. We recommend `scikit-learn` as the primary machine learning library.

6. **Submit your work on eeclass**. No late submission will be accepted (except the pre-registration, see Section 2). The main submission consists of:

Submission	Deadline
A **pre-registration report** in the PDF format, up to **2 pages.** File name: HW2_pre_report_{student_id}.pdf	10/20 11:59 PM
A **final report** in the PDF format, up to **15 pages.** File name: HW2_final_report_{student_id}.pdf	11/3 11:59 PM
An **.ipynb notebook** containing all code needed to reproduce the results reported in your submission. File name: HW2_code_{student_id}.ipynb	
The **AI-use log** document. File name: HW2_ai_use_{student_id}.md	

Materials

The following materials for Homework 2 have been posted on eeclass. Please head over to the platform for full details and instructions.

- The starter notebook `hw2_starter.ipynb`
- The cleaned dataset `processed_tracks.csv`, described in Section 1
- The build script `build_processed_tracks.py` for reference, which reproduces `processed_tracks.csv` from the raw dataset `tracks.csv`
- **Assigned reading** (not covered in class): the slides of *Model Selection and Evaluation*

Section 1: The Data

The file `processed_tracks.csv` contains 5,300 rows and the same 20 columns as `tracks.csv` in homework 1. Each row is now one track (a single `track_id`) and its label `track_genre` is one of following 20 genres:

techno	ambient	comedy	blues	r-n-b
709	481	477	449	422
sleep	punk	jazz	hip-hop	classical
411	344	289	289	231
heavy-metal	bluegrass	reggae	country	grunge
223	205	160	141	118
death-metal	tango	opera	black-metal	salsa
100	82	62	58	49

Rare genres. Throughout this assignment, t**he rare genres are the five smallest genres in the table: death-metal, tango, opera, black-metal, and salsa.** When a question asks for the recall (or precision) of the rare genres, report the value of each of the five and their average.

Here is the procedure to build the `processed_tracks.csv` from `tracks.csv`, you may refer to `build_processed_tracks.py` for more details:

1. Removed the rows with missing metadata and the rows with `time_signature` equals 0 (failed audio analysis).
2. Removed repeated (`track_id`, `track_genre`) rows and only kept the 20 genres listed above.
3. **No duplicate tracks**: Removed rows with identical audio features, and versions of the same recordings. Different recordings of the same composition by different performers are kept as different tracks.
4. Kept **at most three tracks** with the same artist credit in each genre. A prolific artist can still appear many times with different co-artists.
5. Sampled the genres so that their sizes form a long-tail, as in a real catalog.

Reminder: Unless a question says otherwise, use only these 13 audio features as inputs: `danceability, energy, speechiness, acousticness, instrumentalness, liveness, valence, loudness, tempo, key, mode, time_signature, duration_ms`. The values of the audio features are in the original units (e.g., `duration_ms` is in milliseconds).

Section 2: Pre-Registration - Set Up an Evaluation Protocol (5%)

Submit this section report **before 10/20 11:59 PM.**

Your pre-registration is graded on **completeness and specificity**, not on whether your predictions are correct.

- Submitted by *October 20*: Up to **5** points.
- Submitted between *October 20 and November 3*: Up to **2** points.
- Not submitted: Section 2 receives **0** points.

Every number in *Sections 3 and 4* comes from an evaluation protocol. **The goal of this section is to build the protocol before you train any classifiers.**

A test score estimates how a classifier will perform on tracks it has never seen. The boxes below summarize the evaluation methods mentioned in the assigned reading "*Model Selection and Evaluation*". Please make sure you have fully understood the concept before moving on to the implementation.

Training, validation, and test sets

The **training set** is used to fit a model. The **test set** estimates how the final model will perform on new tracks. When learning a model, **pretend that you do not have the test data yet**: if the test labels influence the model in any way, the test score is **biased** upward. This includes every decision made by looking at data, such as choosing k or a tree depth, choosing a decision threshold, selecting features, or fitting the mean and standard deviation of a scaler.

Such decisions are made on a separate **validation set taken from the training data**: fit each candidate on the rest of the training data, compare the candidates on the validation set, choose one and only evaluate the chosen model *once* on the test set.

k-fold cross-validation

A single validation set is small, and the score depends on which tracks happen to be in it. ***k*-fold cross-validation** splits the training data D into k disjoint folds $D_1, \ldots, D_k$ of about **equal size**.

For $j = 1, \ldots, k$, the model $f^{(-j)}$ is trained on $D \setminus D_j$ and scored on D_j, giving a score s_j. The cross-validated estimate and its spread are

$$\bar{s} = \frac{1}{k}\sum_{j=1}^{k} s_j, \qquad \mathrm{sd} = \sqrt{\frac{1}{k-1}\sum_{j=1}^{k}(s_j - \bar{s})^2}$$

Every training track is used for validation exactly once. To compare two settings fairly, evaluate both on the same folds.

Stratified sampling

When a dataset is split at random, **a rare class can be missing from a fold** or appear several times more often than expected. **Stratified** splitting keeps the proportion of every class close to its proportion in the whole dataset, in every fold and in the train/test split.

Leave-one-out cross-validation (LOOCV)

LOOCV is k-fold cross-validation with $k = n$ (the number of training examples), in which every fold holds out a single example. It uses the data most efficiently, which makes it **useful when a dataset is very small**. It requires n model fits, so with thousands of tracks it is slow and gains little.

Nested (internal) cross-validation

If hyperparameters are chosen by cross-validation, the cross-validated score of the chosen setting is slightly **optimistic**, because the same folds were used to make the choice. **Nested cross-validation** avoids this: an outer loop

holds out data for evaluation, and an inner cross-validation inside each outer training part chooses the hyperparameters.

Metrics for many classes

Let y_i be the true genre of track i, $\hat{y}_i$ the predicted genre, n the number of tracks, and C the number of genres. The **accuracy** is

$$\mathrm{Acc} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left[\hat{y}_i = y_i\right]$$

$$P_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FP}_c}, \qquad R_c = \frac{\mathrm{TP}_c}{\mathrm{TP}_c + \mathrm{FN}_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}$$

Where P is precision, R is recall, and F_1 refers to F_1 score. If a denominator is 0 (for example, a genre that is never predicted), the value is defined as 0.

The **macro** average gives every genre the same weight, while the **micro** average pools the counts, so it gives every track the same weight:

$$F_1^{\mathrm{macro}} = \frac{1}{C}\sum_{c=1}^{C} F_{1,c}$$

$$P^{\mathrm{micro}} = \frac{\sum_c \mathrm{TP}_c}{\sum_c(\mathrm{TP}_c + \mathrm{FP}_c)}, \qquad R^{\mathrm{micro}} = \frac{\sum_c \mathrm{TP}_c}{\sum_c(\mathrm{TP}_c + \mathrm{FN}_c)}, \qquad F_1^{\mathrm{micro}} = \frac{2P^{\mathrm{micro}}R^{\mathrm{micro}}}{P^{\mathrm{micro}} + R^{\mathrm{micro}}}$$

When the classes are **imbalanced**, accuracy can hide almost everything about the rare classes.

1. **(5%) Build the protocol** that every later section will use:

    a. **Dataset Partitioning and Reproducibility:**
    Hold out a stratified test set from the full dataset. Specify the exact test split ratio, the number of cross-validation folds, whether the CV folds are stratified, and the fixed random seed(s) used across all splits.

    b. **Leakage-Free Hyperparameter Tuning and Reporting Metrics:**
    Explain how hyperparameters will be selected exclusively on the

training folds (e.g., via inner CV or grid search) without accessing the test set. State which evaluation metrics you will report.

c. **Empirical Justification of Stratification:**
Tabulate the distribution of track counts for rare genres across individual folds under two regimes: *standard random K-Fold versus Stratified K-Fold.* Use these concrete counts to justify **why stratification is necessary for this dataset**.

d. **Trivial Baselines:**
Evaluate **at least two feature-blind baselines** using your protocol (e.g., majority-class predictor and class-prior stratified random guesser). Report both accuracy and macro-F_1 for each. Provide a mathematical explanation based on their formal definitions.

e. **Theoretical Equivalence of Micro-F_1 and Accuracy:**
Compute both micro-averaged F_1 and accuracy of the baseline predictions. Prove analytically and explain conceptually why the two metrics are **identical** in this single-label multiclass setting.

Section 3: Getting to Know Each Classifier (35%)

Each classifier family introduced in class relies on distinct **inductive biases**, with **hyperparameters** controlling its decision dynamics.

In this section, you will systematically **evaluate five classifier families** one at a time under the following protocol:

- Fit all feature transformers, scalers, and other preprocessing steps strictly on the training partition of each cross-validation fold, transforming validation / test folds downstream to prevent data leakage.
- Explain your observed empirical results through the lens of how the underlying algorithm functions, backing your claims directly with your reported quantitative scores.
- Library defaults (e.g., in `scikit-learn`) may not align with theoretical best practices or optimal convergence; explicitly identify, document, and justify any non-trivial default parameters you adjust or rely upon.

- For a hyperparameter that controls how complex a model is (e.g., k in k-NN, the depth of a tree, C and γ of an SVM), plot the training score and the cross-validated score against the hyperparameter.

---

1. **(8%) *k*-Nearest Neighbors**

    a. **Look inside a prediction:**
    Train k-NN on the **standardized features** of the training part of one fold, with a k of your choice. Pick at least three tracks from the validation part of that fold that it gets wrong, including at least one from a rare genre. For each, **list its nearest neighbors** in the training part (genre and **distance**) and how many votes each genre receives. Explain how the vote produced the wrong answer.

    b. **The number of neighbors:**
    On standardized features, vary k from 1 to **at least 100**, plot the training and cross-validated of accuracy and macro-F_1 against k. Explain the shape of the curves, and why the training accuracy at $k = 1$ has the value it has. Also, **which *k* is best by accuracy**, and **which by macro-*F1***? If they differ, explain why from **how the vote works**, support your explanation with the **cross-validated recall of the rare genres** (e.g. their average) at several values of k.

    c. **What does "near" mean:**
    After standardizing the 13 features, append N extra columns of noise drawn independently **from a standard normal distribution $\mathcal{N}(0, 1)$**, so that the noise has the same scale as the standardized features. Use several values of N (e.g., $N = 0, 5, 20, 50$). **Generate the noise once for all tracks**, so that a track keeps the same noise values whether it is in the training or the validation part. **Explain the noise effect from the distance** that k-NN computes. For each track, what fraction of its nearest neighbors k before the noise is added are still among its nearest neighbors after the noise is added?

2. **(9%) Naive Bayes**

    a. **Look inside a prediction:**

Because naive Bayes multiplies the prior π by one Gaussian likelihood per feature, the log of the ratio of the posteriors of two genres A and B is a sum of one term for the prior and one term for each feature:

$$\log \frac{P(A \mid x)}{P(B \mid x)} = \log \frac{\pi_A}{\pi_B} + \sum_{j=1}^{13} \log \frac{\mathcal{N}(x_j; \mu_{Aj}, \sigma_{Aj}^2)}{\mathcal{N}(x_j; \mu_{Bj}, \sigma_{Bj}^2)},$$

Fit the classifier (with its default settings of the library function), **on the standardized features** of the training part of one fold. Pick one track x from the validation part, and the two genres A and B with the highest predicted probability for it. Compute the 14 terms included the prior term from the fitted model's parameters. **Identify which terms decide the prediction**, and relate the largest terms to the means and variances of those features in the two genres.

b. A model that should not care about scale:
Gaussian naive Bayes models each feature **separately**, so **in principle rescaling a feature should not change its predictions**. Compare the classifier **on raw and on standardized features**. If the results differ, find out why from your library's documentation or source code, and explain it with numbers from your data. On standardized features, **vary the smoothing parameter** (`var_smoothing` in `scikit-learn`) over several orders of magnitude, and report how the cross-validated scores and their spread across folds change. If the default performs poorly, find which genre and which feature cause it, and explain why. **The value you choose will be used in the rest of the homework.**

c. When the assumptions are wrong:
Naive Bayes assumes that, within each genre, every feature follows a Gaussian distribution and is **independent** of the others. Show at least one feature that clearly violates the **gaussian assumption** and at least one pair of features that clearly violates the **independence assumption**, each with evidence computed within genres.

d. Confidence and the reliability diagram:
Use the smoothing you chose in (b), and add exact copies of one feature to the inputs (e.g., 1, 3, 5 copies of `energy`). **Report the**

average confidence next to the accuracy and **draw a reliability diagram**, which groups the validation tracks by their confidence (e.g., 0~0.1, 0.1~0.2, …, 0.9~1), and **plot the accuracy of each group against its average confidence**.

** The confidence on a track is the predicted probability of the genre the classifier predicts (the largest posterior probability).*

** For a classifier whose probabilities **can be trusted**, the points on a reliability diagram lie on the diagonal; points below it mean the classifier is more confident than it should be.*

3. (9%) Decision Trees and Their Ensembles

a. Look inside the tree:

Train a tree with a maximum depth of 3 and plot it (which is small enough to read). Describe the feature and threshold of each split in the first two levels, and which genres each split separates. Explain why the tree splits on these first, using **how a split is chosen by measuring the decrease in *impurity***. Report the cross-validated accuracy and macro-F_1 of trees with maximum depth 3, and list the genres that such a small tree never predicts, and explain why.

b. Growing and pruning:

Vary **at least two hyperparameters** that control the size of the tree (e.g., maximum depth, minimum samples per leaf), each over a range wide enough to **go from clear underfitting to clear overfitting**. For each, plot the training and cross-validated scores, and report the number of leaves. Identify **where the tree underfits and where it overfits**.

c. Many trees

Compare **a single fully grown tree, bagging, and a random forest**, and report their cross-validated scores. Measure how much each model's predictions change when it is trained on a different random sample. For example, train each model twice on two different bootstrap samples (e.g., *sampling with replacement*) of the training set, and report the fraction of validation tracks on which the two versions disagree.

Show how the scores of bagging and of the random forest change with the **number of trees from 1 to at least a few *hundred*.** Explain your observations about **why averaging many models helps**, and **compare the random forest with bagging** both in score and in how much their predictions change with **mathematical evidence**.

4. **(9%) Support Vector Machines**

   a. **The two hyperparameters of an RBF kernel:**
   Evaluate an **RBF-kernel** SVM on standardized features over a grid of ***C* and γ**, each spanning at least four orders of magnitude (e.g., C from 0.1 to 1000 and γ from 0.001 to 10). Show the training and cross-validated scores (e.g., as heatmaps), and **identify the regions where the model underfits and where it overfits**, explain both from what C and γ control, and choose a setting.

   b. **Support vectors:**
   With $\gamma = 0.01$ and several values of C, fit the SVM on the whole training set and report the fraction of training tracks for each genre that become support vectors. Relate the fraction of each genre to **how well that genre is recognized (via cross-validated F_1)** and to **how many training tracks** it has. Can the fraction of support vectors be read as a measure of how much a genre overlaps with the others?

Section 4: Beyond Accuracy (10%)

A classifier is built for a purpose, and a single accuracy number rarely tells you whether it serves that purpose. The boxes below define the two-class metrics that you need in this section.

The two-class confusion matrix and its rates

In a two-class problem, one class is called *positive* (here, "hip-hop") and the other *negative*. The confusion matrix counts the four possible outcomes:

	predicted positive	**predicted negative**
true positive class	TP (true positives)	FN (false negatives)
true negative class	FP (false positives)	TN (true negatives)

With $N = \mathrm{TP} + \mathrm{FN} + \mathrm{FP} + \mathrm{TN}$:

$$\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{N}$$

$$\mathrm{TPR}\ (\text{recall, sensitivity}) = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$$

$$\mathrm{TNR}\ (\text{specificity}) = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$$

$$\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP} + \mathrm{TN}} = 1 - \mathrm{TNR}$$

$$\mathrm{PPV}\ (\text{precision}) = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad \mathrm{NPV} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FN}}$$

F_1 score

F_1 combines precision and recall into one number through their harmonic mean:

$$F_1 = \frac{2 \cdot \mathrm{PPV} \cdot \mathrm{TPR}}{\mathrm{PPV} + \mathrm{TPR}} = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP} + \mathrm{FP} + \mathrm{FN}}$$

It is high only when both precision and recall are high, and it does not use TN.

Scores, thresholds, the ROC curve, and its AUC

Most classifiers output a score $s(x)$ (e.g., a predicted probability of the positive class) and predict "positive" when $s(x) \geq t$ for a threshold t. Each threshold gives its own TPR(t) and FPR(t). The **ROC curve** plots TPR(t) against FPR(t) as t moves from $+\infty$ (nothing is positive, the point (0, 0)) to $-\infty$ (everything is positive, the point (1, 1)). The area under it is

$$\mathrm{AUC} = \int_0^1 \mathrm{TPR}\; d(\mathrm{FPR})$$

Equivalently, AUC is the probability that a randomly chosen positive example x^+ receives a higher score than a randomly chosen negative example x^-. On n_+ positive and n_- negative examples it is computed as

$$\mathrm{AUC} = \frac{1}{n_+ n_-} \sum_{i \in \mathrm{pos}} \sum_{j \in \mathrm{neg}} u_{ij}, \qquad u_{ij} = \begin{cases} 1 & s(x_i) > s(x_j), \\ \frac{1}{2} & s(x_i) = s(x_j), \\ 0 & s(x_i) < s(x_j). \end{cases}$$

A random score gives AUC = 0.5, and a perfect ranking gives 1. For many classes, a **one-versus-rest** AUC treats one genre as positive and all others as negative, using the predicted probability of that genre as the score.

Choosing a threshold

The default threshold (e.g., a probability of 0.5) is rarely the right one. A threshold should be chosen for what the application needs, for example "find at least 80% of the positives" (a minimum TPR) or "at least half of the flagged tracks must be correct" (a minimum precision). Like any other decision made by looking at data, the threshold must be chosen **without the test set**.

1. **(4%) Different numbers, different conclusions:**

    a. Using the best setting you found for each family in Section 3 (state by which metric you chose it), report the cross-validated accuracy and macro-F_1 of every family. How far apart are the two numbers for each family, and why?

b. For your best model (state by which metric), trained on the training set, showed how its **multi-class confusion matrix** on the test set.

c. Which genres does the model fail on, and which genres does it confuse them with? Compute a **one-versus-rest ROC-AUC** for every genre from the model's predicted probabilities on the test set, and compare it with the genre's F_1 on the test set. Can a genre have a high AUC and still a low F_1?

2. (6%) A hip-hop detector.

The team also wants a detector that answers "hip-hop" or "not hip-hop" for each track.

a. Build detectors from **at least two families** that output a continuous score (e.g., a probability or a decision value). Train them on **two-class labels** (hip-hop or not), and choose their hyperparameters without the test set. State which metric you use to choose them, and why.

b. On the test set, report the accuracy of a detector that never answers "hip-hop". For each detector at its default threshold (a probability of 0.5, or a decision value of 0), report the confusion matrix, accuracy, TPR, TNR, PPV, NPV, and F_1 .

c. Plot the ROC and precision-recall curves, and report the area under each. Which numbers show that a detector is useful, and which do not?

d. A curator will listen to every track the detector flags, and wants to find **at least 80%** of the hip-hop tracks. For one of your detectors, **choose a threshold** that meets this need from out-of-fold predicted scores on the training set. **On the test set, report whether the recall reaches 80% and what it costs** (e.g., tracks to listen to and hip-hop tracks missed per 1,000 tracks).

Section 5: AI-Use Log

AI tools are permitted in this assignment. The purpose of this log is to document how you used them and how you verified their outputs.

Submit an AI-use log written in the markdown file. If you used any AI tool during the assignment, briefly describe:

1. **What you used AI for.**
   List the main parts of the assignment for which you used AI, such as writing or debugging code, explaining a concept, suggesting an analysis, interpreting a result.

2. **How you checked the output.**
   For each major use, briefly explain how you verified that the AI-generated output was correct or appropriate. For example, you might compare it with lecture material, inspect the code, run an additional experiment, or check the result against the data.

If you did not use any AI tools, simply state that in the log.

You remain responsible for all code, analyses, figures, and explanations in your submission. **You should be able to explain any AI-assisted work during the demo session.**

A **5-point deduction** will be applied for **missing submissions or conspicuously low-effort responses** (e.g., merely writing "I used ChatGPT").

Grading Policy

Report (50%)

Tier	Description	Percentage
4	Provides evidence, explanation, and decisions, along with a discussion of limitations or alternative interpretations.	100%
3	Provides evidence, explanation, and decisions, with coherent logic linking all three.	80%
2	Presents results or figures, but the explanation is	50%

	weak or disconnected from the evidence.	
1	Only presents results without explanation, or the methodology is clearly flawed.	20%
0	No answer provided / Blank.	0%

To prevent grader bias, each section is assigned to a dedicated TA who evaluates all students' submissions for that specific part. Rest assured that your work will be assessed with a fair and consistent standard.

Demo (50%)

The demo will take place on **Thursday, November 12, in the evening**, together with the HW1 demo. Further details, including the exact time, location, and demo format, will be announced later.

Feel free to ask questions! For homework-related issues, please post on the eeclass discussion forum, and the TAs will reply promptly. For personal matters or private inquiries, you can reach the TA at george930502@gapp.nthu.edu.tw.